

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF SOUTHERN PHILIPPINES

CS 414 - 4B - ACTIVITY 1 (Part 1): Havel-Hakimi Algorithm

Group Members:

Besa, Casiño, Legaspi, Fabria, Pacuribot, Rejas, Sorongon

Problem No. 5

A social network analyst wants to map interactions among 6 club members with the target degree sequence: $S_5 = (5, 4, 3, 2, 1, 3)$.

a. Determining Whether the Sequence is Graphical

Preliminary Check:

Number of vertices (n) = 6

Sum of degrees = $5 + 4 + 3 + 2 + 1 + 3 = 18$ (even, necessary condition satisfied)

Maximum degree = $5 = n - 1$ (does not exceed the maximum possible degree)

S_5 is given unsorted. It is sorted to $(5, 4, 3, 3, 2, 1)$ before applying the algorithm, since Havel Hakimi requires non-increasing order at every step. Sorting reorders the working list only — each vertex keeps the degree assigned to it by the given sequence.

Applying the Havel-Hakimi Algorithm:

Step 1:

Sort in non-increasing order: $(5, 4, 3, 3, 2, 1)$

Remove the first element (5).

Subtract 1 from the next 5 elements:

$(4-1, 3-1, 3-1, 2-1, 1-1) = (3, 2, 2, 1, 0)$

No negative values. Proceed.

Step 2:

Sort in non-increasing order: $(3, 2, 2, 1, 0)$

Remove the first element (3).

Subtract 1 from the next 3 elements:

$(2-1, 2-1, 1-1, 0) = (1, 1, 0, 0)$

No negative values. Proceed.

Step 3:

Sort in non-increasing order: $(1, 1, 0, 0)$

Remove the first element (1).

Subtract 1 from the next 1 element:

$(1-1, 0, 0) = (0, 0, 0)$

No negative values. Proceed.

Step 4:

Remaining sequence: (0, 0, 0)

All elements are zero.

Conclusion: The sequence $S_5 = (5, 4, 3, 2, 1, 3)$ IS GRAPHICAL. A simple graph with this exact degree sequence can be constructed.

b. Graph Construction

Vertex Assignment (by position in the given sequence $S_5 = (5, 4, 3, 2, 1, 3)$): $v_1 = \text{degree } 5$ $v_3 = \text{degree } 3$ $v_5 = \text{degree } 1$
 $v_2 = \text{degree } 4$ $v_4 = \text{degree } 2$ $v_6 = \text{degree } 3$

Working order for the algorithm (sorted by degree, labels unchanged):

$v_1(5), v_2(4), v_3(3), v_6(3), v_4(2), v_5(1)$

Edge List (derived by tracing the algorithm):

From Step 1 (v_1 connects to all other vertices):

$v_1-v_2, v_1-v_3, v_1-v_6, v_1-v_4, v_1-v_5$

From Step 2 (v_2 connects to the next 3 highest-degree vertices): $v_2-v_3, v_2-v_6, v_2-v_4$

From Step 3 (v_3 connects to the next 1 highest-degree vertex): v_3-v_6

Total edges: 9

Adjacency Matrix:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	[0	1	1	1	1	1
v_2	[1	0	1	1	0	1
v_3	[1	1	0	0	0	1
v_4	[1	1	0	0	0	0
v_5	[1	0	0	0	0	0
v_6	[1	1	1	0	0	0

Verification:

v_1 : connected to $v_2, v_3, v_4, v_5, v_6 \rightarrow$ degree 5 OK

v_2 : connected to $v_1, v_3, v_4, v_6 \rightarrow$ degree 4 OK

v_3 : connected to $v_1, v_2, v_6 \rightarrow$ degree 3 OK

v_4 : connected to $v_1, v_2 \rightarrow$ degree 2 OK

v_5 : connected to $v_1 \rightarrow$ degree 1 OK

v_6 : connected to $v_1, v_2, v_3 \rightarrow$ degree 3 OK

Sum of degrees = $5 + 4 + 3 + 2 + 1 + 3 = 18 = 2 \times 9$ edges

The graph satisfies the given degree sequence $S_5 = (5, 4, 3, 2, 1, 3)$.

Graph Drawing:

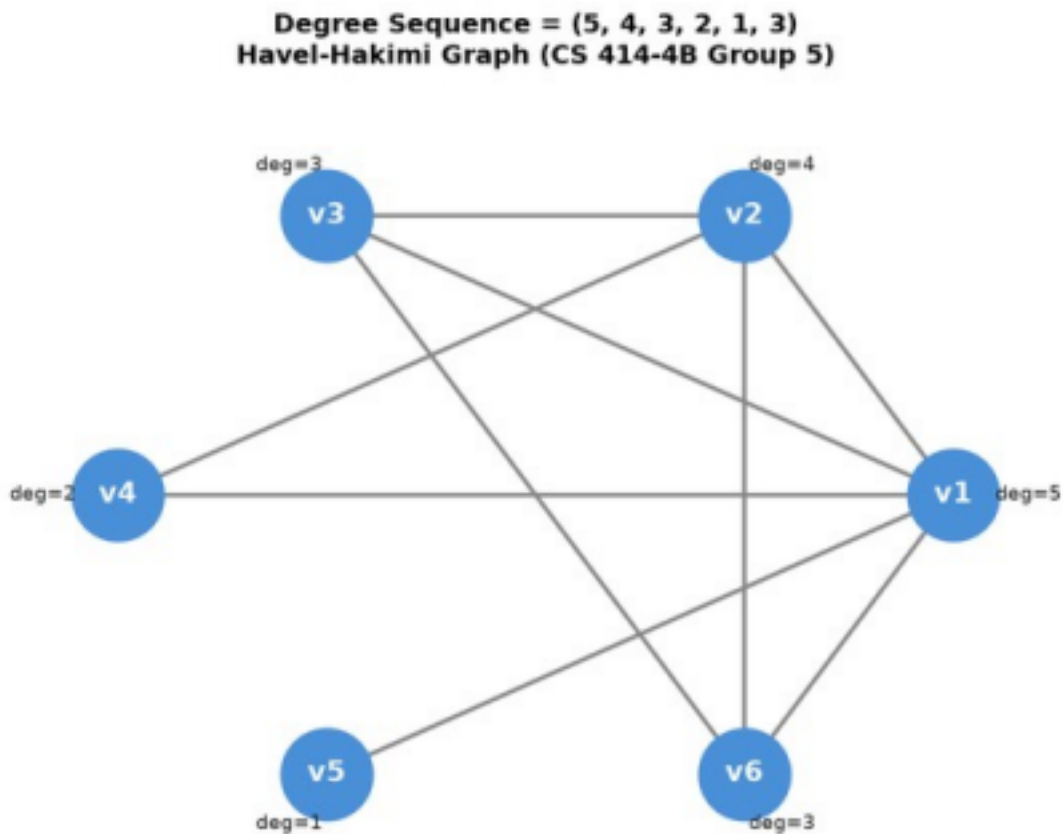


Figure 1. Constructed graph for $S_5 = (5, 4, 3, 2, 1, 3)$.
Each vertex is labeled with its identifier and its degree.

Source Code of implementation:

```
# =====
# CS 414 - 4B | Activity 1 (Part 2): Havel-Hakimi Algorithm
# Problem No. 5 — stripped version (final graph only, no step-by-step figures)
# Degree Sequence:  $S_5 = (5, 4, 3, 2, 1, 3)$ 
#
# Group Members:
#   Wency Casiño      Ken Charles Besa      Birky Pacuribot
#   Kerby Fabria     Carl Rejas      Cj Legaspi
#   Charles Sorongon
# =====

from pathlib import Path

import networkx as nx
import matplotlib.pyplot as plt

# Save images next to this file, not in whatever directory the script was run from
OUTPUT_DIR = Path(__file__).resolve().parent

"""
Determines if a degree sequence is graphical using the Havel-Hakimi algorithm.
```

The algorithm repeatedly:

1. Sorts the sequence in non-increasing order.
2. Removes the largest degree (d) from the front.
3. Subtracts 1 from the next d elements.
4. Repeats until all zeros (graphical) or a negative appears (not graphical).

Returns True if graphical, False otherwise.

Also prints each step of the reduction to the console.

"""

```
def is_graphical(sequence):
    seq = sorted(sequence, reverse=True)
    step = 1

    print(f" Starting sequence (sorted): {seq}")

    while True:
        # Remove trailing zeros — they don't affect graphicality
        seq = [d for d in seq if d > 0]

        # Base case: all elements are zero → sequence is graphical
        if not seq:
            print(f"\n All degrees reduced to zero.")
            return True

        # Fail: first degree exceeds the number of remaining vertices
        d = seq[0]
        remaining = len(seq) - 1
        if d > remaining:
            print(f"\n Step {step}: Degree {d} > available nodes ({remaining}) → FAIL")
            return False

        print(f"\n Step {step}: Sequence = {seq}")
        print(f" Take largest degree: {d}")
        print(f" Subtract 1 from next {d} elements: {seq[1:d+1]}")

        # Subtract 1 from the next d elements, keep the rest unchanged
        new_seq = [seq[i] - 1 for i in range(1, d + 1)] + seq[d + 1:]

        # Fail: subtraction produced a negative degree
        if any(x < 0 for x in new_seq):
            print(f" Result has a negative value → FAIL")
            return False

        new_seq_sorted = sorted(new_seq, reverse=True)
        print(f" Remaining (re-sorted): {new_seq_sorted}")
        seq = new_seq_sorted
        step += 1
```

```
def build_graph(sequence, node_labels):
    G = nx.Graph()          #Graph Builder
    G.add_nodes_from(node_labels)

    # Pair each node label with its target degree: [(degree, label), ...]
    indexed = list(zip(sequence, node_labels))

    while True:
        # Sort pairs by degree, highest first
        indexed = sorted(indexed, key=lambda x: x[0], reverse=True)

        # Drop nodes whose degree requirement is already satisfied
```

```

indexed = [(d, v) for d, v in indexed if d > 0]

# Stop when no unsatisfied degrees remain
if not indexed:
    break

d, v = indexed[0] # Node with the highest remaining degree

# Connect v to the next d highest-degree nodes, then remove v
for i in range(1, d + 1):
    neighbor_deg, neighbor = indexed[i]
    G.add_edge(v, neighbor)
    indexed[i] = (neighbor_deg - 1, neighbor) # Reduce neighbor's remaining degree

indexed = indexed[1:] # Remove v — its degree is now fully satisfied

return G

def draw_graph(G, sequence, node_labels):
    """
    Renders and saves a visual plot of the constructed graph using matplotlib.
    Nodes are labelled by their identifier and annotated with their degree.

    Parameters:
        G : the constructed NetworkX graph
        sequence : original degree sequence (for the title)
        node_labels: list of node names
    """
    plt.figure(figsize=(8, 6))

    # Use a circular layout so that all nodes are evenly spaced
    pos = nx.circular_layout(G)

    # Draw edges first (behind nodes)
    nx.draw_networkx_edges(G, pos, edge_color='#888888', width=2)

    # Draw nodes
    nx.draw_networkx_nodes(G, pos, node_color='#4A90D9', node_size=1800)

    # Label each node with its name
    nx.draw_networkx_labels(G, pos, font_size=13, font_color='white', font_weight='bold')

    # Annotate each node with its degree, placed just outside the node
    degree_labels = {node: f"deg={G.degree(node)}" for node in G.nodes()}
    label_pos = {node: (x * 1.18, y * 1.18) for node, (x, y) in pos.items()}
    nx.draw_networkx_labels(G, label_pos, labels=degree_labels,
                           font_size=9, font_color='#222222')

    plt.title(
        f"Degree Sequence = {tuple(sequence)}\n"
        "Havel-Hakimi Graph (CS 414-4B Group 5)",
        fontsize=12, fontweight='bold', pad=15
    )
    plt.margins(0.15) # room for the deg= labels drawn outside the nodes
    plt.axis('off')
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "problem5_graph.png", dpi=150)
    plt.show(block=False) # detached: the window stays open, the script continues
    plt.pause(0.1) # gives the window a moment to draw
    print("\n Graph saved as 'problem5_graph.png'")

```

```

def print_results(G, node_labels):
    """
    Prints the edge list and the verified degree of every vertex to the console.

    Parameters:
        G : the constructed NetworkX graph
        node_labels: list of node names (for ordered output)
    """
    print("\n Edge list:")
    for u, v in sorted(G.edges()):
        print(f" {u} — {v}")

    print("\n Verified degree of each vertex:")
    for node in node_labels:
        print(f" {node}: degree = {G.degree(node)}")


def read_sequence(): # Input validation and Normalization
    """
    Reads a degree sequence typed by the user, re-prompting until it is valid.

    Accepts numbers separated by spaces or commas. A blank line falls back to
    S5 from the activity, so the graded run needs no typing.

    Returns:
        list of non-negative integers, or None if input ends (Ctrl-D)
    """
    while True:
        try:
            raw = input(" Enter degree sequence (blank = S5 -> 5, 4, 3, 2, 1, 3): ").strip()
        except EOFError:
            return None

        if not raw:
            return [5, 4, 3, 2, 1, 3]

        try:
            sequence = [int(token) for token in raw.replace(",", " ").split()]
        except ValueError:
            print(" Invalid input: use whole numbers only, e.g. 5 4 3 2 1 3\n")
            continue

        if not sequence:
            print(" Invalid input: the sequence is empty.\n")
            continue

        if any(d < 0 for d in sequence):
            print(" Invalid input: a degree cannot be negative.\n")
            continue

        return sequence


def ask_next_action():
    """
    Asks whether to analyse another sequence or quit.

    Returns:
        True to run again, False to exit
    """
    while True:
        print("\n" + "-" * 60)

```

```

print(" [1] Enter another sequence")
print(" [2] Exit")
try:
    choice = input(" Choose: ").strip()
except EOFError:
    return False

if choice == "1":
    print()
    return True
if choice in ("2", ""):
    return False
print(" Please enter 1 or 2.")

```

```
def main():
```

```
    """
```

```
    Entry point. Loops over user-supplied degree sequences. For each one:
```

- a. Havel-Hakimi graphicality check (with step-by-step console output)
- b. Graph construction + edge list + degree verification
- c. Final graph plot via matplotlib (detached window)

```
    """
```

```

print("=" * 60)
print(" CS 414-4B | Activity 1 | Problem 5")
print("=" * 60)

```

```
while True:
```

```

    sequence = read_sequence()
    if sequence is None:
        break

```

```
    node_labels = [f"v{i + 1}" for i in range(len(sequence))]
```

```

    print(f"\n Input sequence: {sequence}")
    print(f" Number of vertices (n) = {len(sequence)}")
    print(f" Sum of degrees = {sum(sequence)}")
    print("=" * 60)

```

```

# — Part a: Check graphicality —————
print("\n[ Part a ] Havel-Hakimi Reduction Steps\n")
graphical = is_graphical(list(sequence))

```

```

if graphical:
    print("\n Result: The sequence IS graphical. A simple graph exists.")

```

```

# — Part b: Build graph and report —————
print("\n" + "=" * 60)
print("[ Part b ] Graph Construction & Verification\n")
G = build_graph(list(sequence), node_labels)
print_results(G, node_labels)

```

```

# — Part c: Visualise the final graph —————
print("\n" + "=" * 60)
print("[ Part c ] Rendering Graph Plot\n")
draw_graph(G, sequence, node_labels)

```

```

else:
    print("\n Result: The sequence is NOT graphical.")
    print(" The network CANNOT be constructed with this degree sequence.")

```

```

if not ask_next_action():
    break

```

```
print("\n Done. Close the plot window to finish.")  
plt.show() # blocks so the detached window stays on screen
```

```
if __name__ == "__main__":  
    main()
```